

Density Mesh Face-Contract Specification

Normative ownership of raw extraction limits, visual targets, and standalone final limits

mdstats

2026-07-22

Contents

1 Purpose and status	1
2 Normative ownership	1
3 Public contracts	2
4 Three distinct face quantities	2
4.1 Raw extraction limit	2
4.2 Visual target	2
4.3 Standalone final limit	2
5 Modes	3
5.1 Standalone mode	3
5.2 Scene-controller mode	3
6 Evaluation semantics	3
7 Deterministic regression fixtures	3
8 Input constraints	3
9 Failure semantics	4
10 Focused validation	4

1 Purpose and status

Module: `mdstats.plotting.density_mesh_contracts`

Status: normative and implemented in `mdstats 0.19.76a0`; consolidated in `0.19.77a0`.

This module owns the meaning and validation of face-count limits for one density shell. It exists because the historical `max_mesh_faces` option mixed three different responsibilities and caused avoidable failures in scene rendering.

2 Normative ownership

The module owns:

- the immutable `DensityMeshFaceContract`;
- the immutable `DensityMeshFaceReport`;
- validation of raw, target, and standalone-final face counts;
- conversion of a final face count into an auditable report;
- the distinction between a visual-target miss and a terminal failure.

It does not own:

- marching-cubes extraction;
- mesh simplification;
- scene-wide target allocation;
- browser-budget fitting;
- Plotly serialization.

Those responsibilities belong respectively to `density_sparse_mesh`, `density_mesh_simplify`, `density_scene_budget`, `density_scene_fit`, and `density_render_budget/framework_dynamics`.

3 Public contracts

```
DensityMeshFaceContract(  
    raw_extraction_face_limit: int | None,  
    visual_target_faces: int | None,  
    standalone_final_face_limit: int | None,  
    mode: Literal["standalone", "scene_controller"],  
    metadata: Mapping[str, JSONScalar],  
)
```

```
DensityMeshFaceReport(  
    final_face_count: int,  
    contract: DensityMeshFaceContract,  
    visual_target_met: bool | None,  
    visual_target_overage_faces: int,  
    standalone_final_limit_met: bool | None,  
)
```

The schema identifiers are:

```
mdstats.density-mesh-face-contract.v1  
mdstats.density-mesh-face-report.v1
```

4 Three distinct face quantities

4.1 Raw extraction limit

`raw_extraction_face_limit` is a computational-safety limit. It protects memory and wall time during contour extraction and initial geometry assembly. The runtime-derived resource limit is authoritative; an explicit contract may only reduce it:

$$F_{\text{raw,resolved}} = \min(F_{\text{runtime}}, F_{\text{contract}}).$$

Exceeding this limit is terminal because the requested extraction cannot be completed safely under the declared resources.

4.2 Visual target

`visual_target_faces` is a preferred final count for one canonical shell. It is a fitting target, not a safety limit. Exceeding it produces

```
report.requires_scene_refit is True
```

and must not raise merely because the target was missed.

4.3 Standalone final limit

`standalone_final_face_limit` is a terminal limit for callers that request one mesh without a scene controller. The historical default remains 250,000 faces for this standalone mode only.

5 Modes

5.1 Standalone mode

```
DensityMeshFaceContract.standalone(  
    final_face_limit=250_000,  
    raw_extraction_face_limit=None,  
    visual_target_faces=None,  
)
```

The final shell is rejected when it exceeds `standalone_final_face_limit`.

5.2 Scene-controller mode

```
DensityMeshFaceContract.scene_controller(  
    raw_extraction_face_limit=runtime_limit,  
    visual_target_faces=allocated_target,  
)
```

This mode requires `standalone_final_face_limit=None`. A target miss is retained for `density_scene_fit`; only raw-work violations remain terminal at the shell level.

6 Evaluation semantics

```
evaluate_density_mesh_face_contract(final_face_count, contract)  
require_density_mesh_face_contract(final_face_count, contract)
```

Evaluation is side-effect free. Requirement raises only when a terminal contract is violated. The following implications are normative:

Condition	Standalone	Scene controller
raw extraction exceeds limit	fail	fail
final count exceeds visual target	report	report and refit
final count exceeds standalone final limit	fail	not applicable
final browser scene exceeds budget	not owned	handled after scene fitting

7 Deterministic regression fixtures

The package retains count-only fixtures for the historical failures:

- 582,375 faces against the historical 250,000 standalone limit;
- 301,838 scene faces against a 300,000 browser limit;
- 314,640 scene faces against a 300,000 browser limit.

The first verifies standalone failure and scene-controller routing. The latter two belong to `density_scene_fit` and must enter closed-loop fitting rather than fail before fitting.

8 Input constraints

- Counts are integers and nonnegative or positive as specified by each field.
- `scene_controller` requires a positive `visual_target_faces`.
- `scene_controller` forbids `standalone_final_face_limit`.
- `standalone` requires a positive final limit.
- Metadata must be JSON compatible and immutable after construction.

9 Failure semantics

Use `GraphStyleError` for invalid option values, `GraphAdapterError` for inconsistent serialized state, and `GraphComplexityError` for a terminal standalone or raw-work violation.

A visual-target miss is never represented as an exception.

10 Focused validation

Tests must cover:

- both constructors;
- runtime raw-limit resolution;
- target-miss reporting without failure;
- standalone terminal failure;
- JSON round trips;
- the 582,375-face regression in both modes;
- integration with dense and sparse mesh paths.